

CS & IT ENGINEERING



Algorithms

Graph Algorithms

Lecture No.- 02



By- Aditya sir

Recap of Previous Lecture



Topic

Graph Traversals

Topic

DFS/DFT

BFS

Topics to be Covered



Topic

DFS

Topic

1. Directed graph

DAG

Topological Sort

ABOUT ME

Hello, I'm Aditya.

1. Represented college as the first Google DSC Ambassador.
2. The only student from the batch to secure an internship at Amazon.
3. Appeared for GATE during BTech and secured AIR 60 in GATE in very first attempt
- City topper
4. Had offer from IIT Bombay and IISc Bangalore to join the Masters program
5. Joined IIT Bombay for my 2 year Masters program in Data Science
6. Published multiple research papers in well known conferences along with the team
7. Received the prestigious excellence in Research award from IIT Bombay for my Masters thesis
8. Completed my Masters with an overall GPA of 9.36/10. Joined Dream11 as a Data Scientist.
9. Have mentored around 10k working professionals in field of Data Science and Analytics
10. Have been mentoring GATE aspirants to secure a great rank in limited time.
11. Have got around 27.5K followers on LinkedIn where I share my insights and guide students and professionals.



By- Aditya sir



Topic : Graph Algorithms

DFS in directed graphs:

Terminologies:-

Whenever we perform DFS on a directed graph, it can lead to the different types of edges listed ~~before~~ below:

1. Tree Edge:

Tree edge are those edges that are present in the DFS spanning tree tree/forest:

The edges that are not present in DFS spanning tree/forest can be 3 types:

1. Forward
2. Backward
3. Cross



Topic : Graph Algorithms

2. Forward edge:-

It is the edge that is going ~~from~~ ^{from} a node to its non-child ~~discontents~~ ^{descendant}.

3. Back/Backward edge:

This is an edge from a node to its ~~acnes for~~ ^{ancestor}.

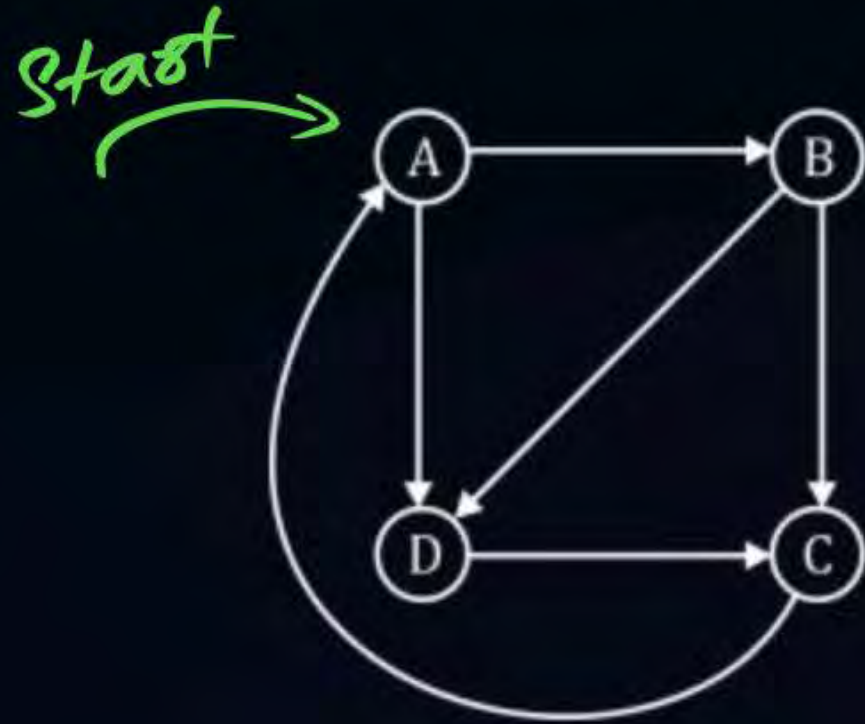
4. Cross edge:-

This is the edge from one node to some other node that is neither its ~~acnes for~~ ^{ancestor} ~~not its discontnet~~ ^{nor its descendant}.



Topic : Graph Algorithms

E.g.1. Given $G(V,E)$

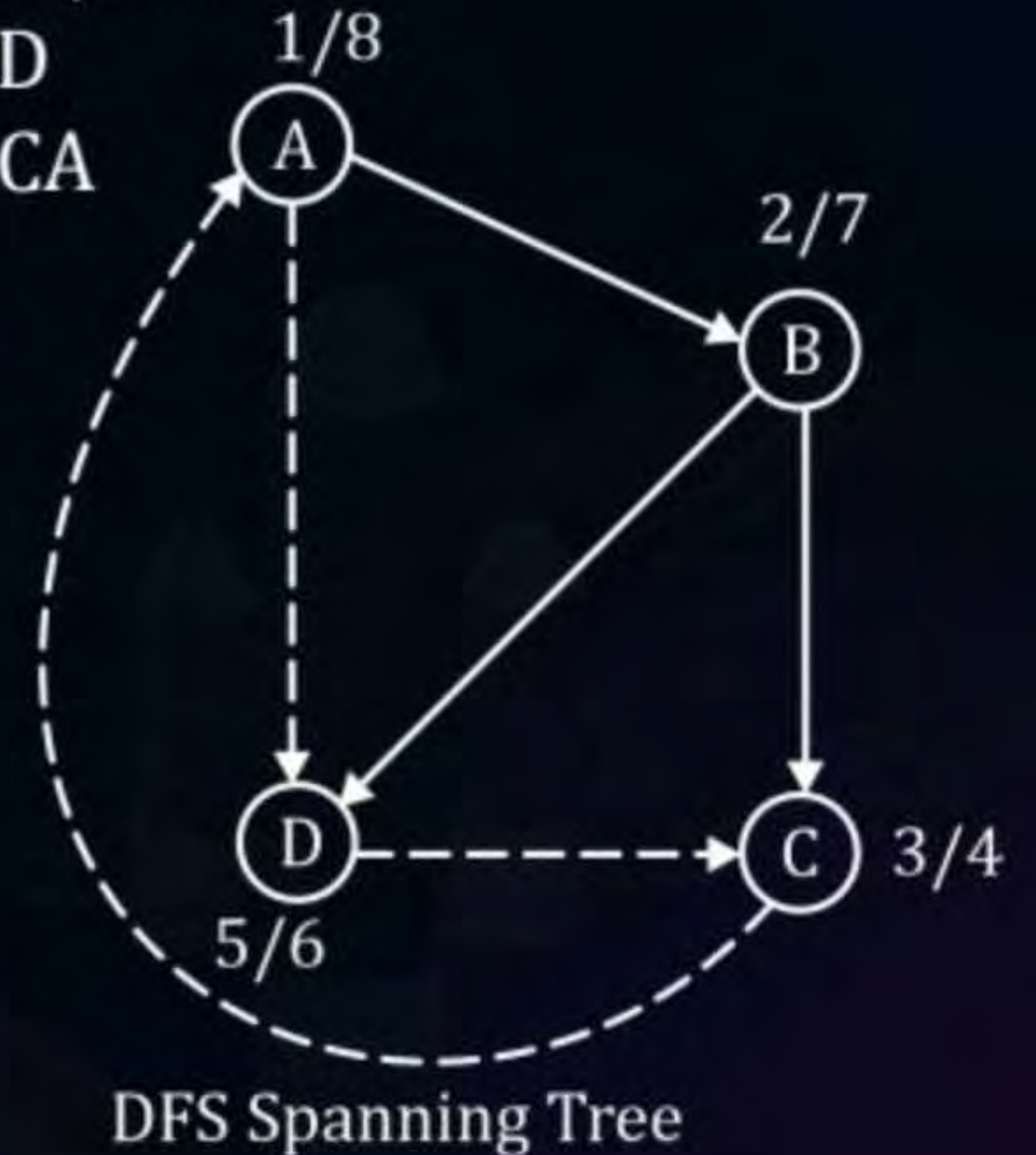


Apply DFS starting at A-

1. DFS Sorting

Types of edges

1. Tree edges:-AB, BC, BD
2. Forward edge:-AD
3. Backward edge:-CA
4. Cross edge:-DC





Topic : Graph Algorithms

E.g.2. 1. $D \rightarrow C$

D: 4: 5/6

C: V: 3/4

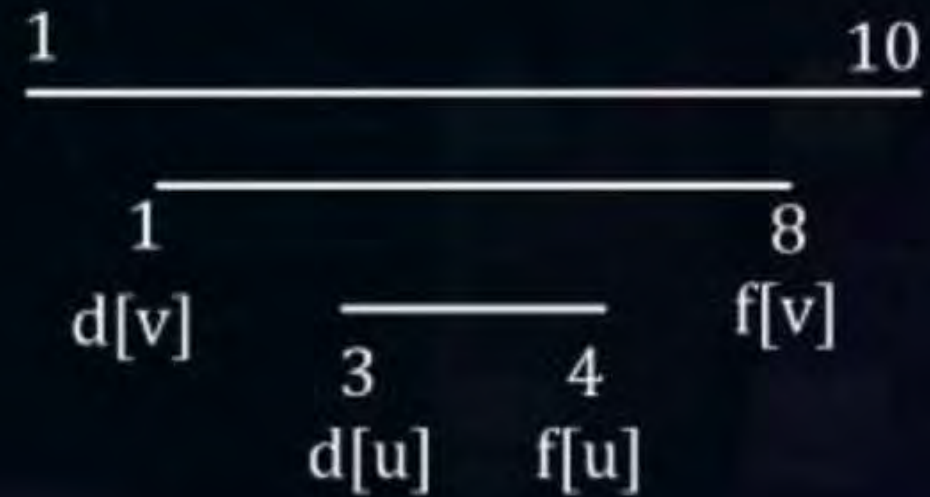
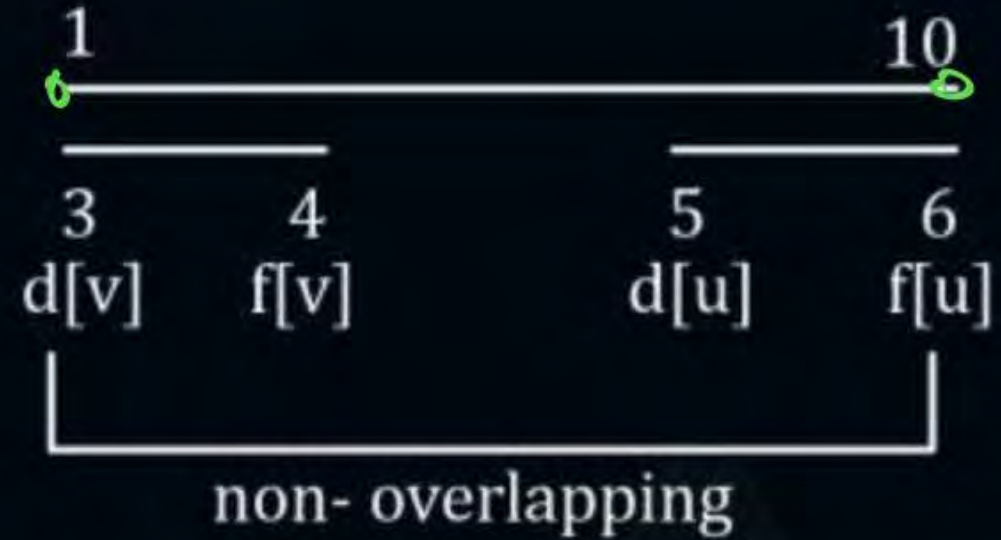
2. $u \rightarrow v$

$C \rightarrow A$

C: d[a] /f[u]: 3/4

C: d[v] /f[v]: 1/8

$u \rightarrow v$ back edge





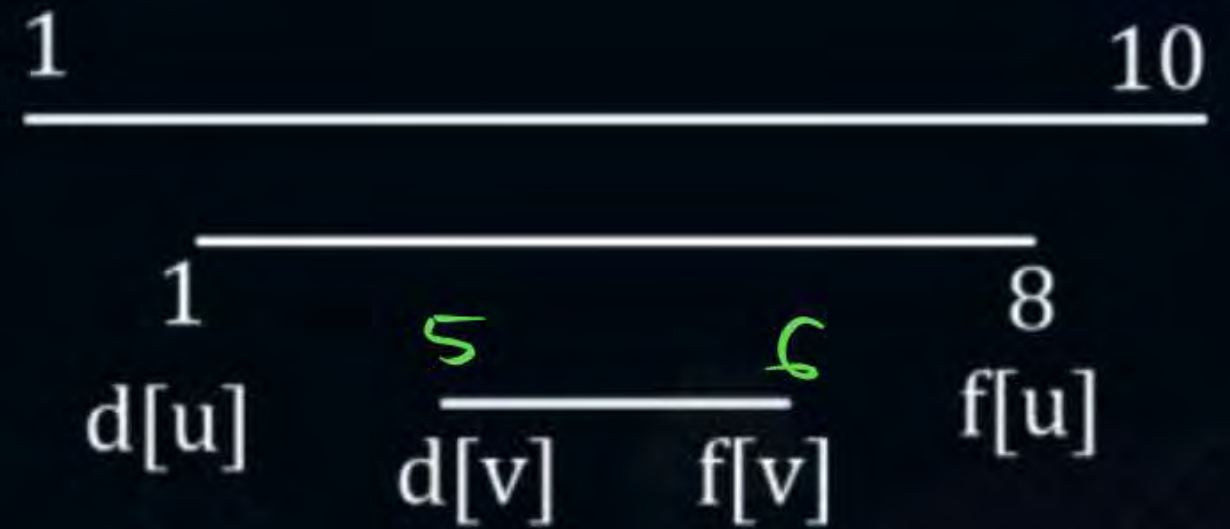
Topic : Graph Algorithms

E.g.3.

u: A: $d[u] / f[u]: 1/8$

v: D: $d[v] / f[v]:$ ~~5/8~~ $5/6$

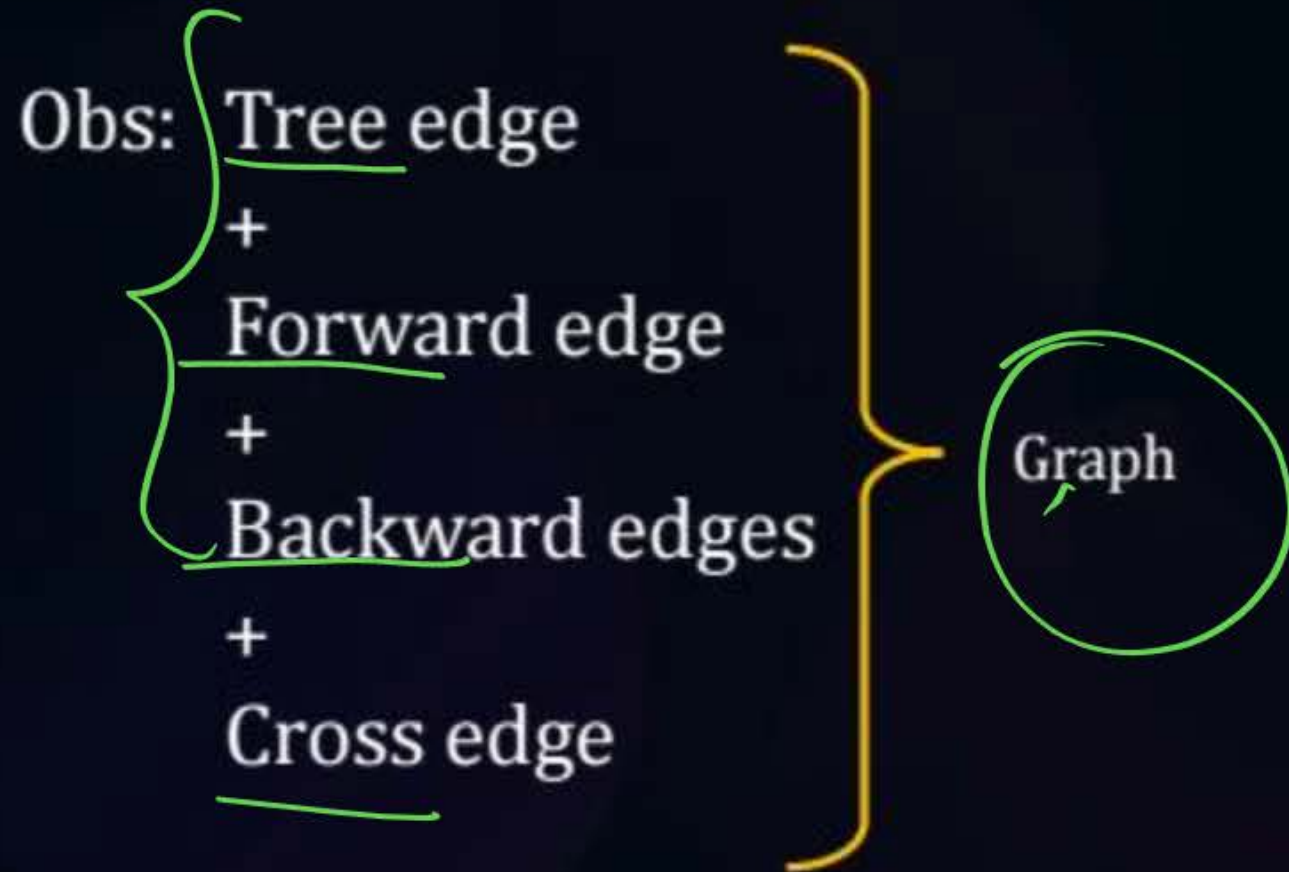
$u \rightarrow v$: forward edge.





Topic : Graph Algorithms

Note. For same graph for same starting vertex different DFS spanning Trees are possible and hence, the types of edges can also be different.

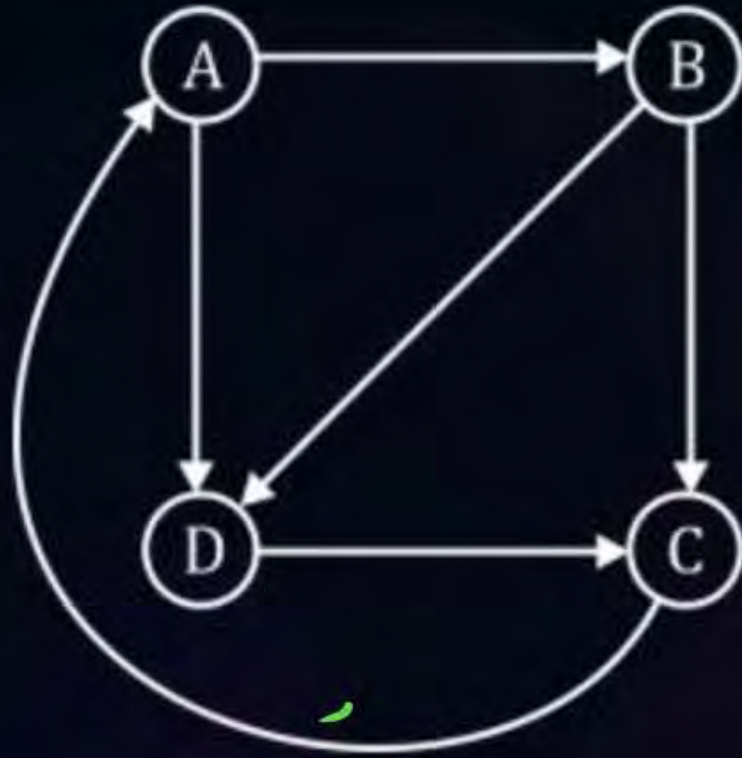




Topic : Graph Algorithms

E.g. 4.

Given $G(V, E)$

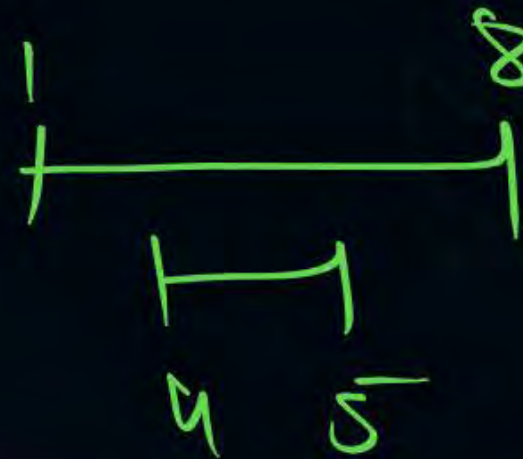


Apply DFS starting at A.

2. DFS Sorting at A

Types of edges

1. Tree edges:- AB, BD, DC
2. Forward edge:- AD, BC
3. Backward edge:- CA
4. Cross edge:- None



DFS Spanning Tree



Topic : Graph Algorithms

$d(u)$
 $f(u)$

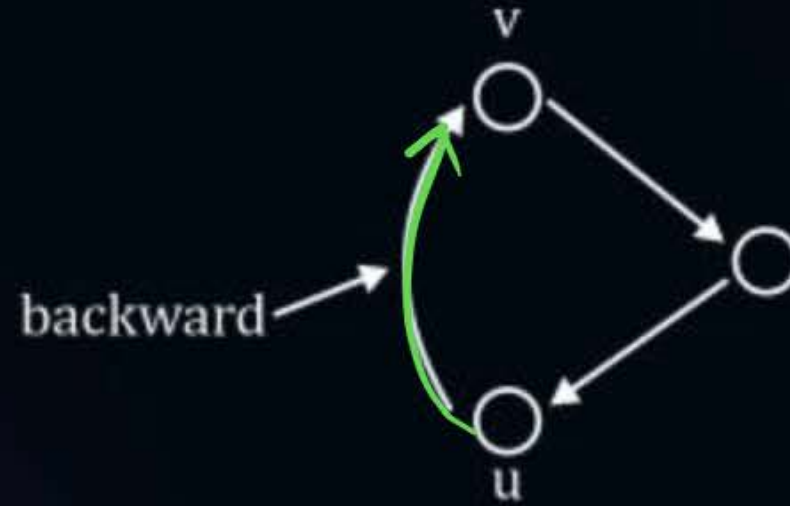
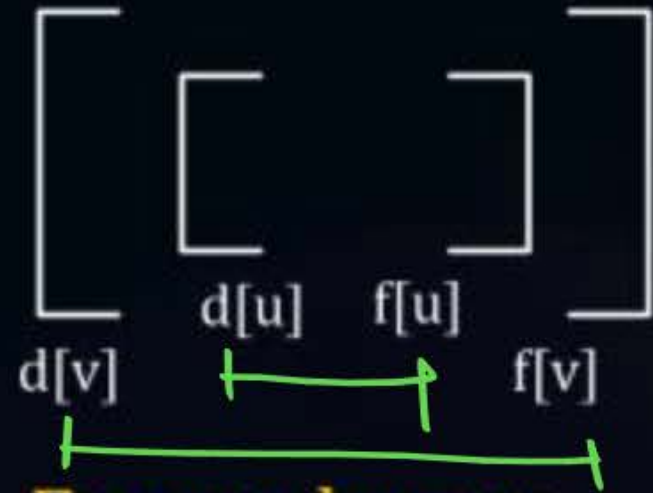
In any Depth-First Search of a (directed or undirected) graph $G = (V, E)$, for any two vertices u and v exactly one of the following three conditions holds:

- I. The intervals $[d[u], f[u]]$ and $[d[v], f[v]]$ are entirely disjoint, and neither u nor v is a descendant of the other in the depth-first forest, $\rightarrow$ cross edge
- ✓ II. The interval $[d[u], f[u]]$ is contained entirely within the interval $[d[v], f[v]]$, and u is a descendant of v in a depth-first tree, or $\rightarrow$ Back edge
- ✓ III. The interval $[d[v], f[v]]$ is contained entirely within the interval $[d[u], f[u]]$, and v is a descendant of u in a depth-first tree.

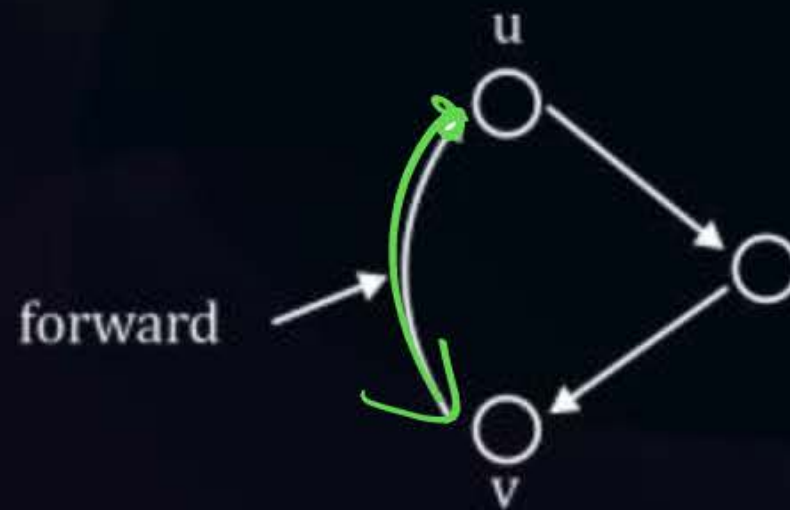
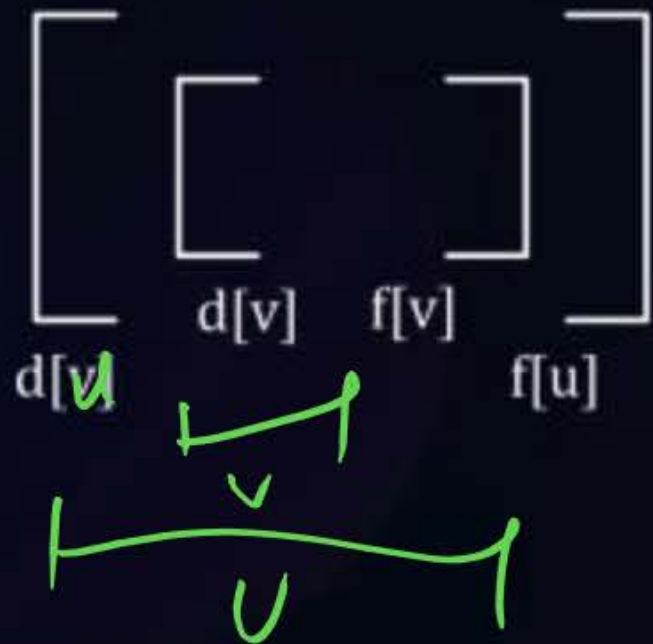


Topic : Graph Algorithms

2. Backward $u \rightarrow v$:



3. Forward $u \rightarrow v$



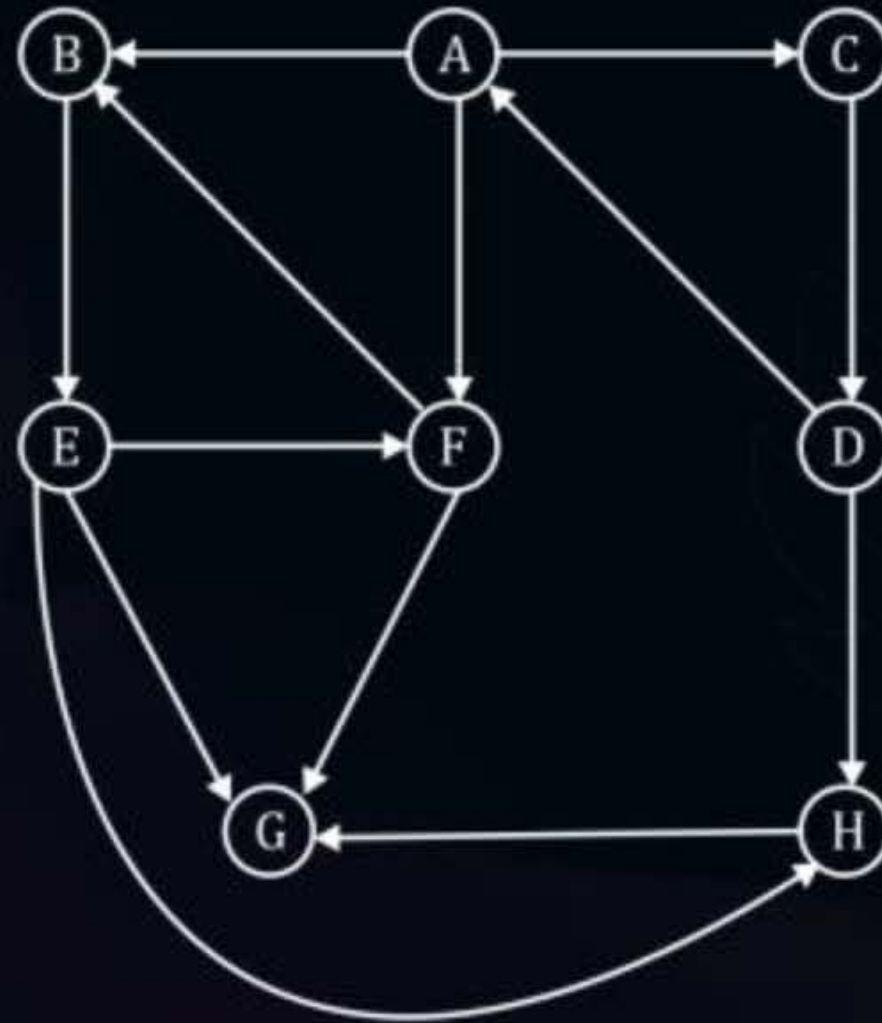


Topic : Graph Algorithms

Test: End to end practice:

Give: $G(V,E)$

DFS starting
A





Topic : Graph Algorithms

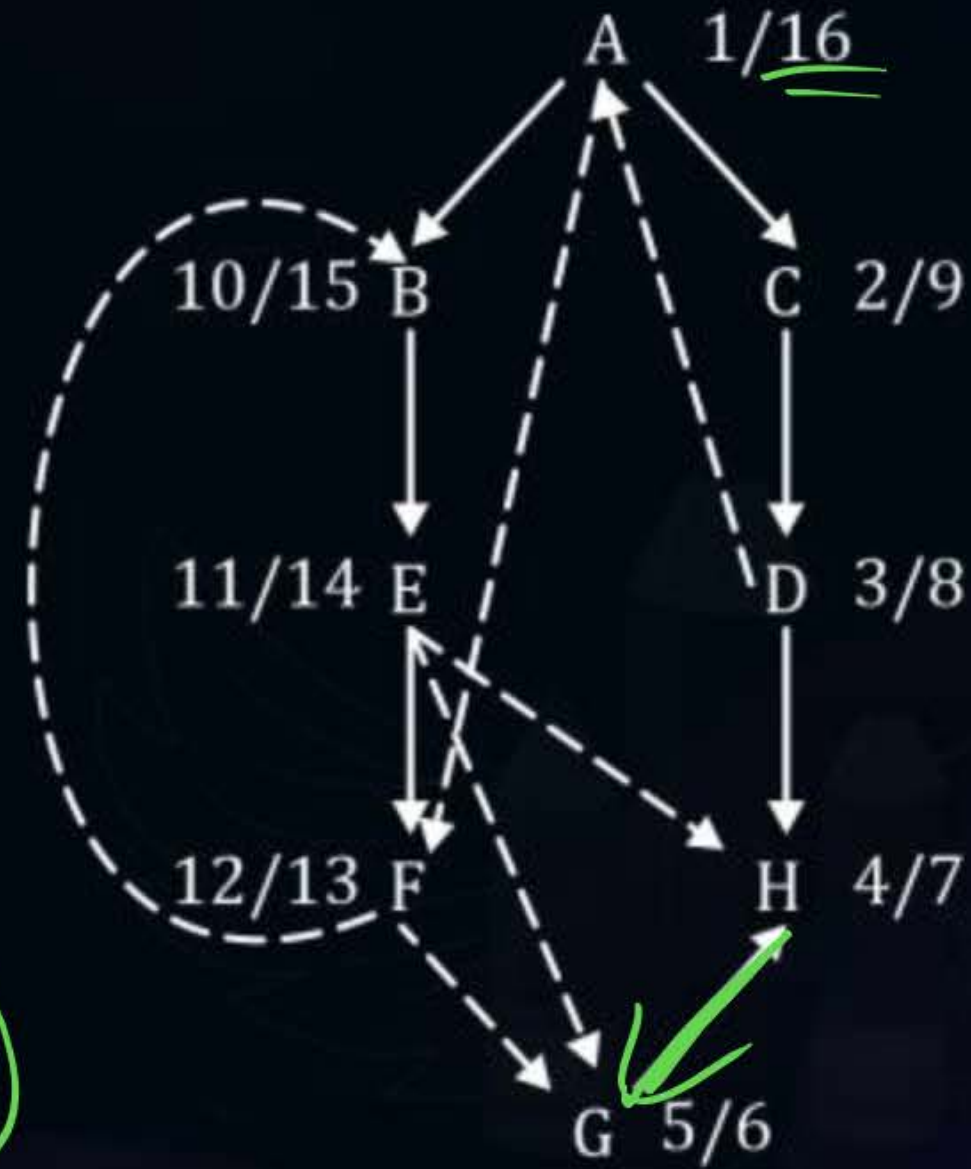
DFS starting at A

Type of edges:

1. Tree edges: AC, CD, DH, HG AB, BE, EF.
2. Forward edges: AF
3. Backward edges: DA, FB
4. Cross edge: FG, ~~EG~~, EH

$$8 \times 2 = 16$$

EG

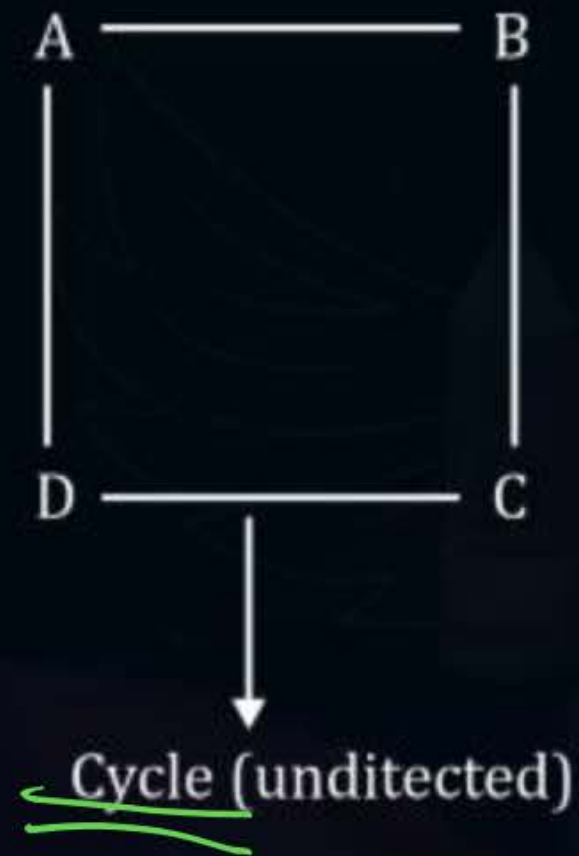
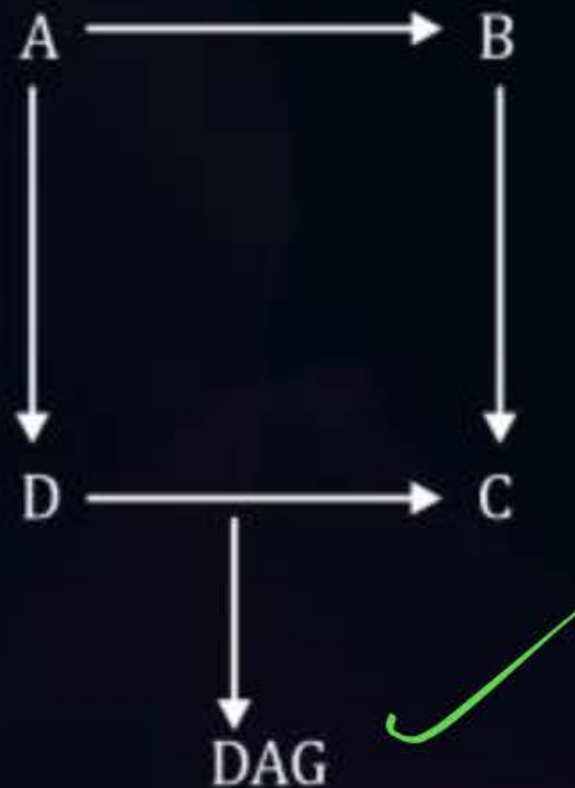




Topic : Graph Algorithms

DFS in directed a cyclic graph: (DAG)

DAG: Directed graph without any cycle in it.





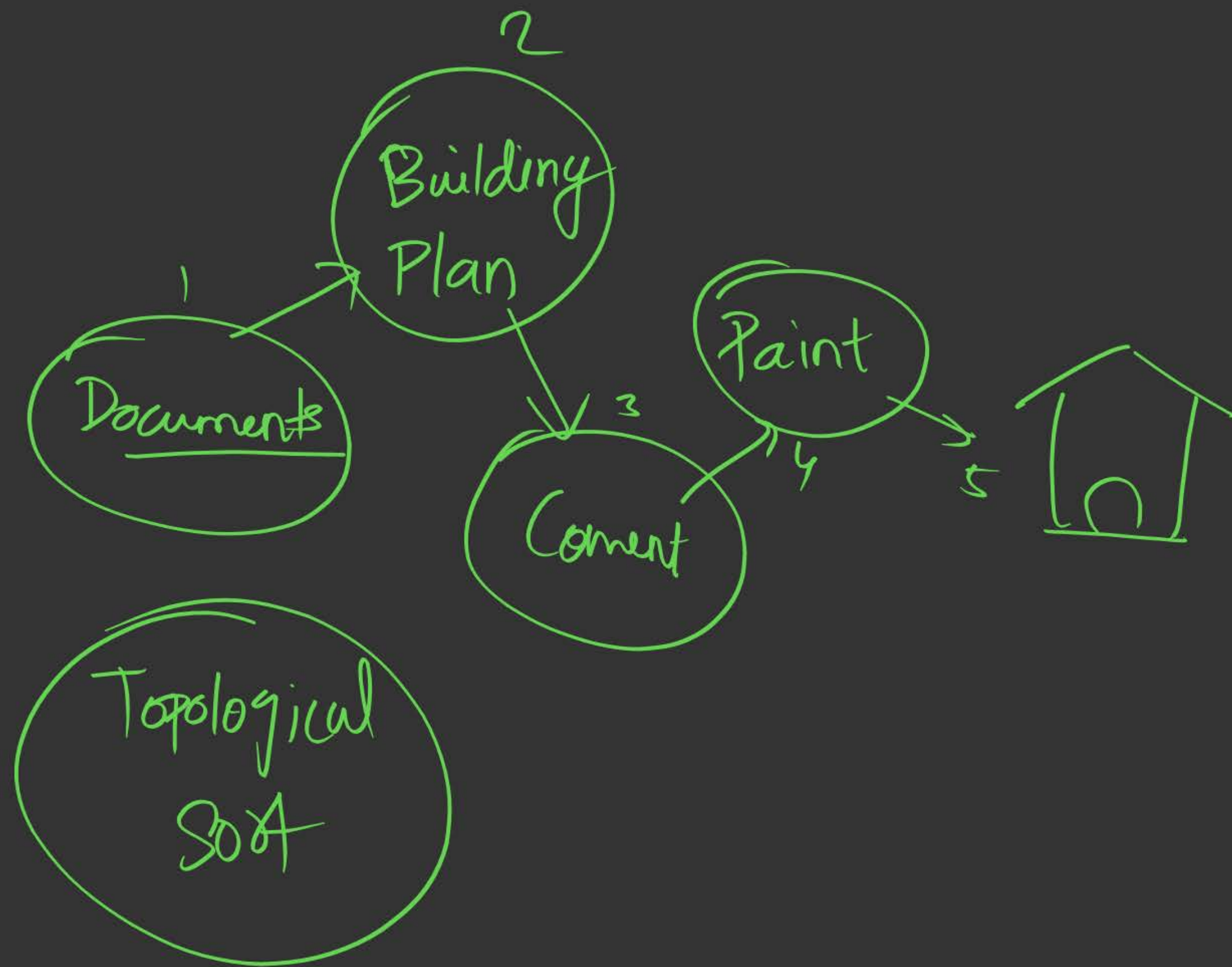
Topic : Graph Algorithms

Topological Sorts: Application of DFS on a directed acyclic graph DAG

Ordering

Definition:

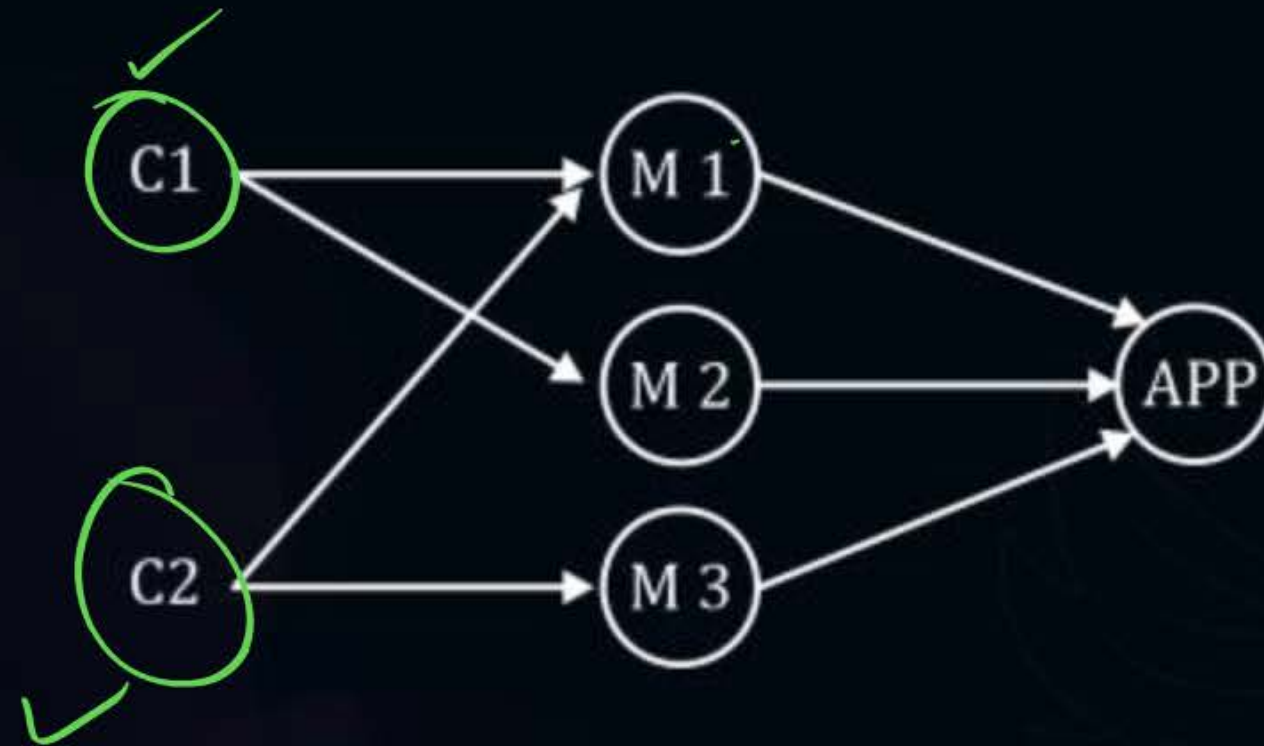
It is the linear order of vertices/nodes that represent some activities that has some precedencies.





Topic : Graph Algorithms

E.g.6.

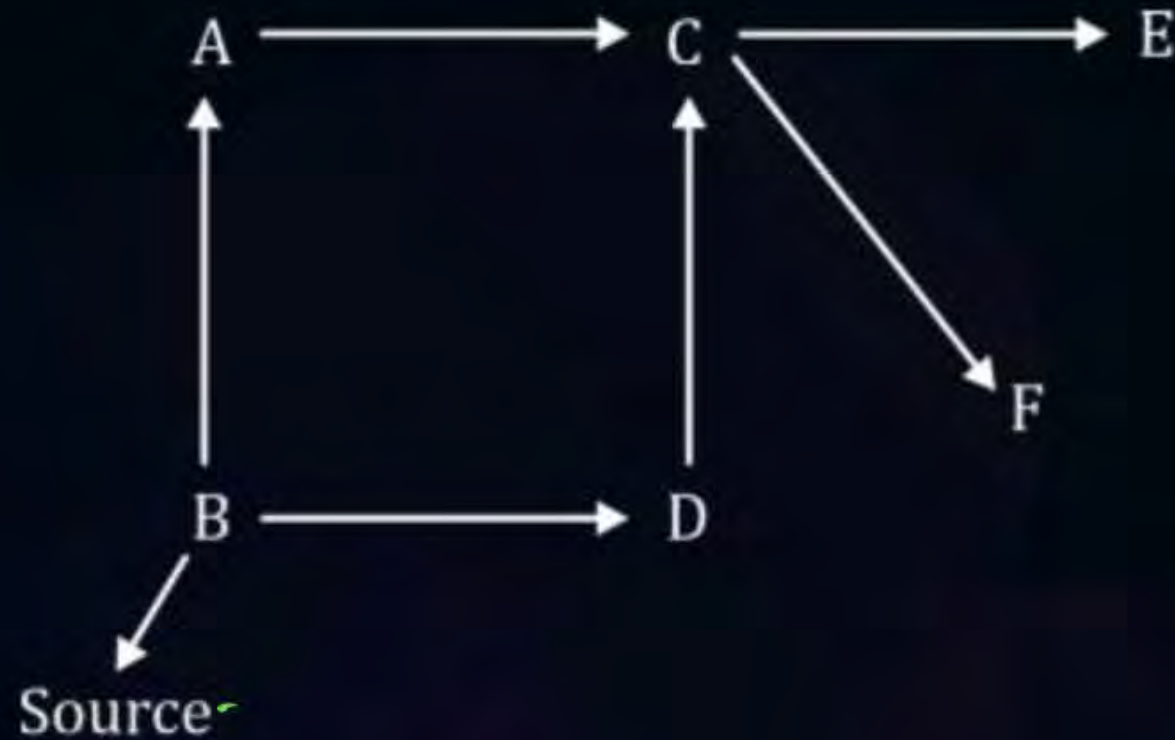




Topic : Graph Algorithms

E.g.7. Given a DAG

DAG



Topological sort

Source → Sink

Sink: Outdegree (no. of outgoing edge = 0)

Source: (no. of incoming edges = 0)

Note:- We visit a node only when all of its precedencies nodes are visits

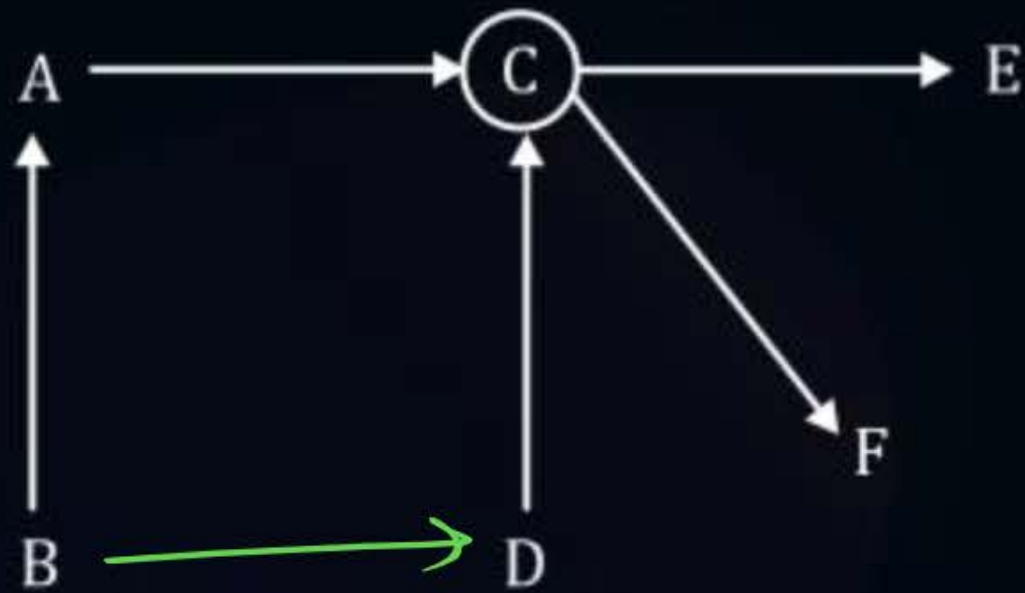
E.g.8. D is precedence of C.



Topic : Graph Algorithms

Finding all possible Topological orderings

Given DAG





Topic : Graph Algorithms

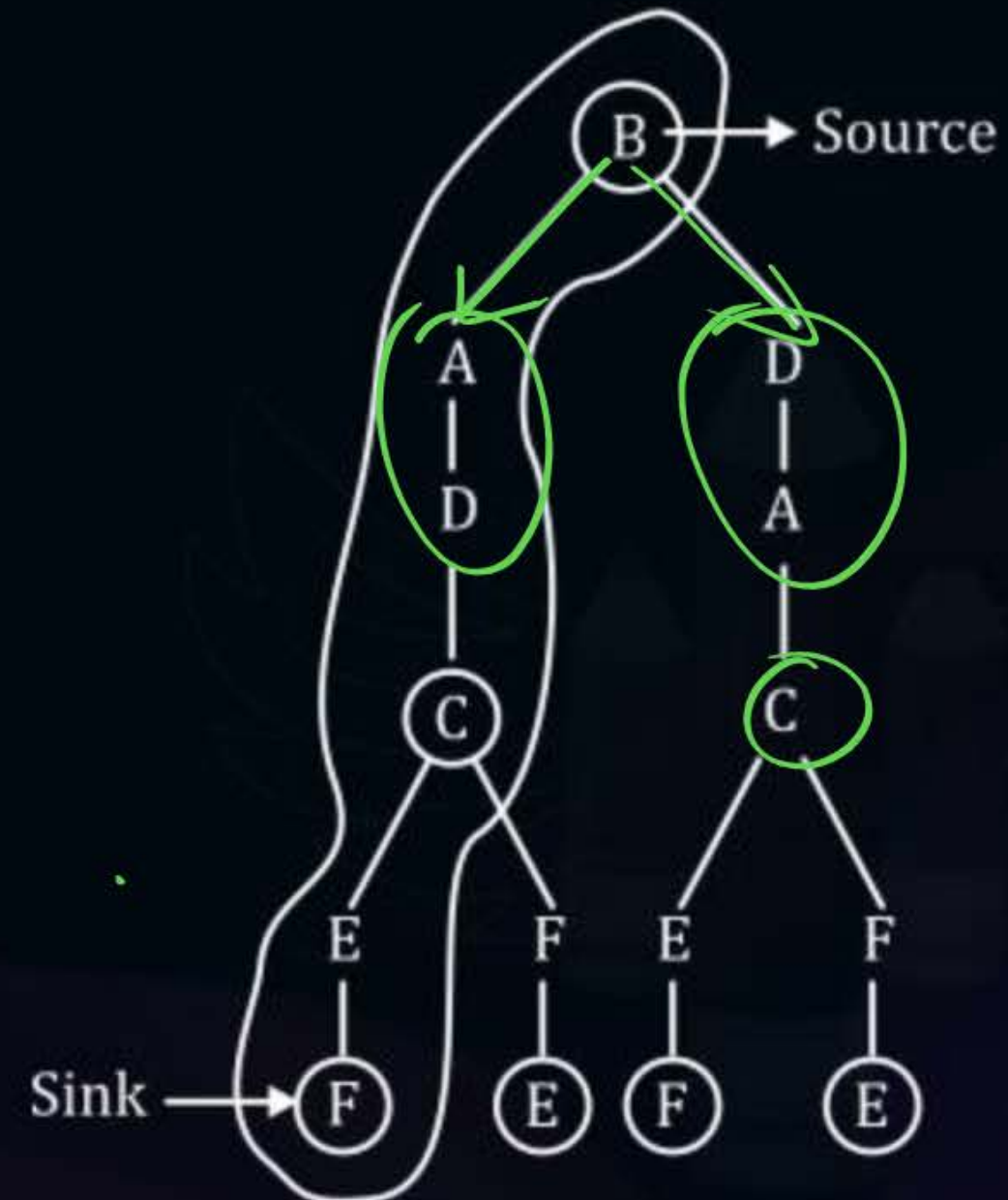
#Q. Every branch from source $\rightarrow$ Sink will give 1 possible topological order.

A BADCEF ✓

B BADCFE ✓

C BDACEF ✓

D BDACFE ✓





Topic : Graph Algorithms

Algorithm for Topological sort using DFS:

Algo Topological sort (G)

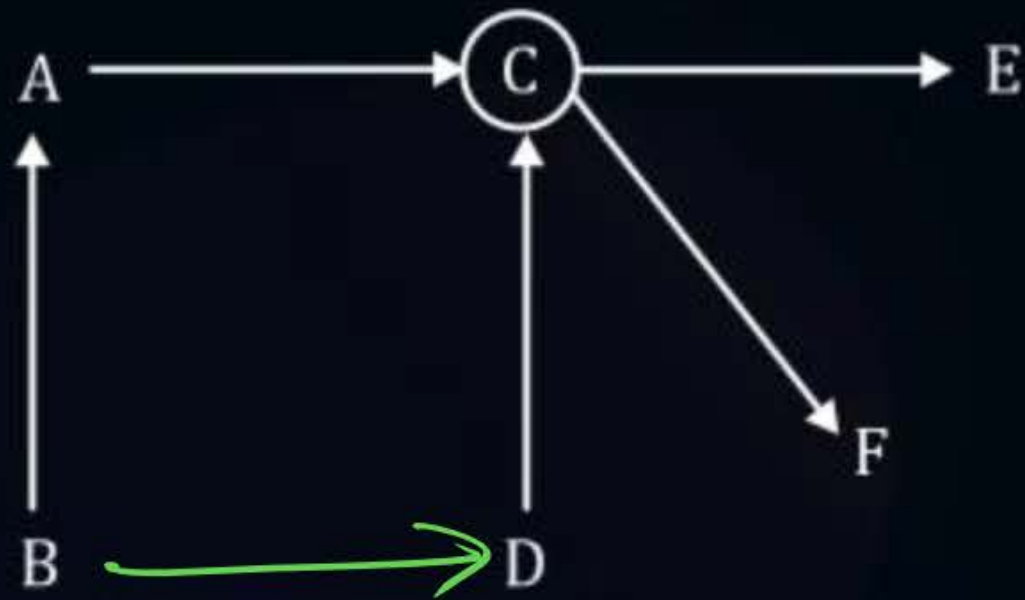
1. Apply DFS starting at any vertex (V): DFS (V)
2. Arrange all the vertices /node obtained in step/ DFS ^{traversal} ~~travels~~ in the descending/decreasing order of their finishing times to get a valid topological sort ordering.



Topic : Graph Algorithms

Walkthrough/ Dry Run of Prev. Algo.

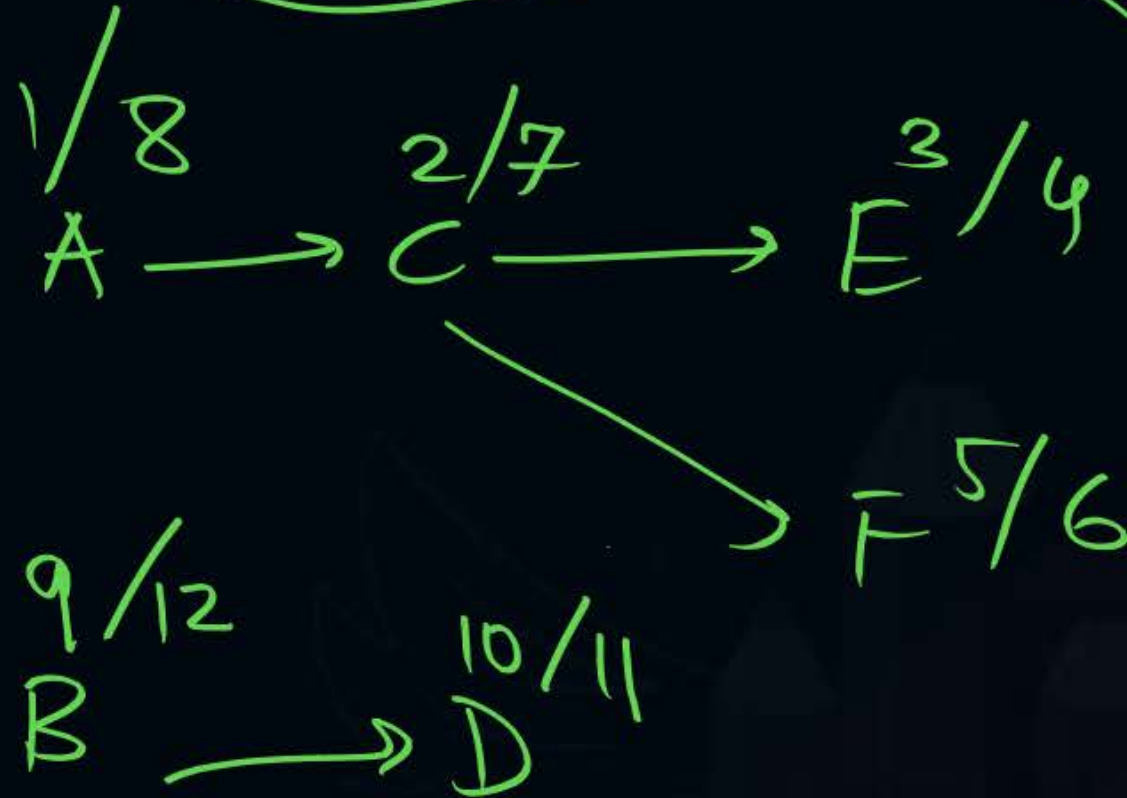
DAG:



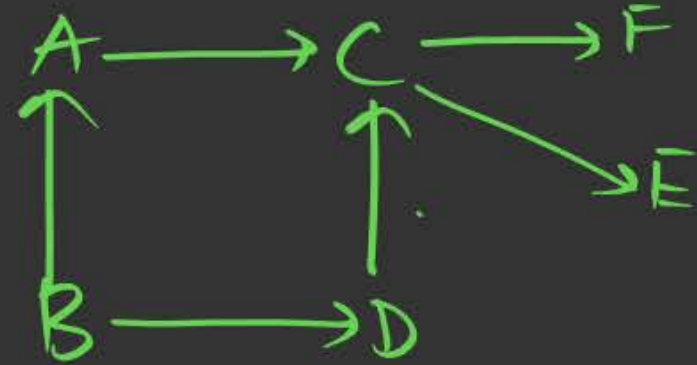
Finishing times

$A \rightarrow 8$ $C \rightarrow 7$ $E \rightarrow 4$
 $B \rightarrow 12$ $D \rightarrow 11$ $F \rightarrow 6$

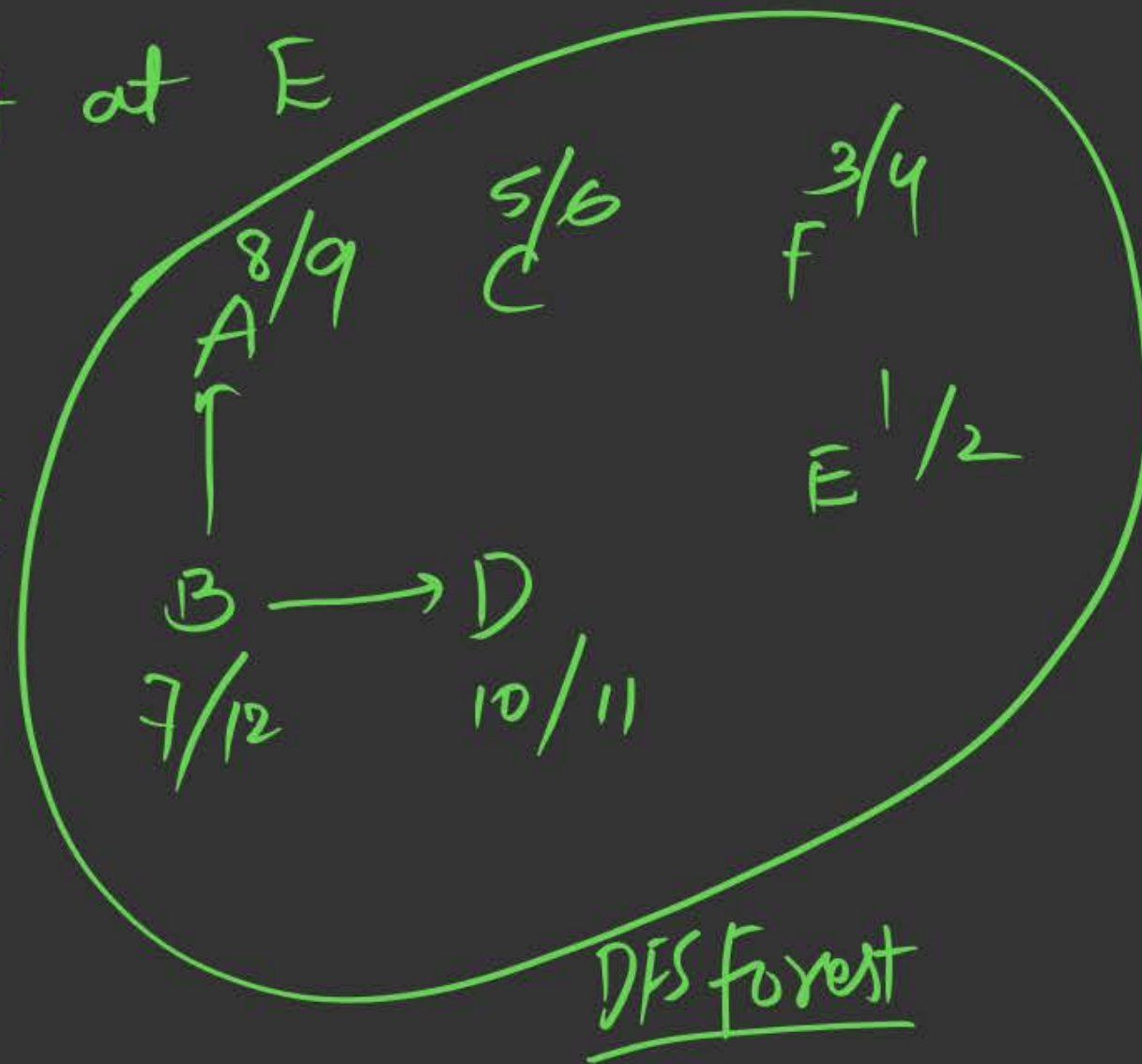
start
A



BDACFE



Start at E



A → 9 E → 2

B → 12 F → 4

C → 6

D → 11

B D A C F E

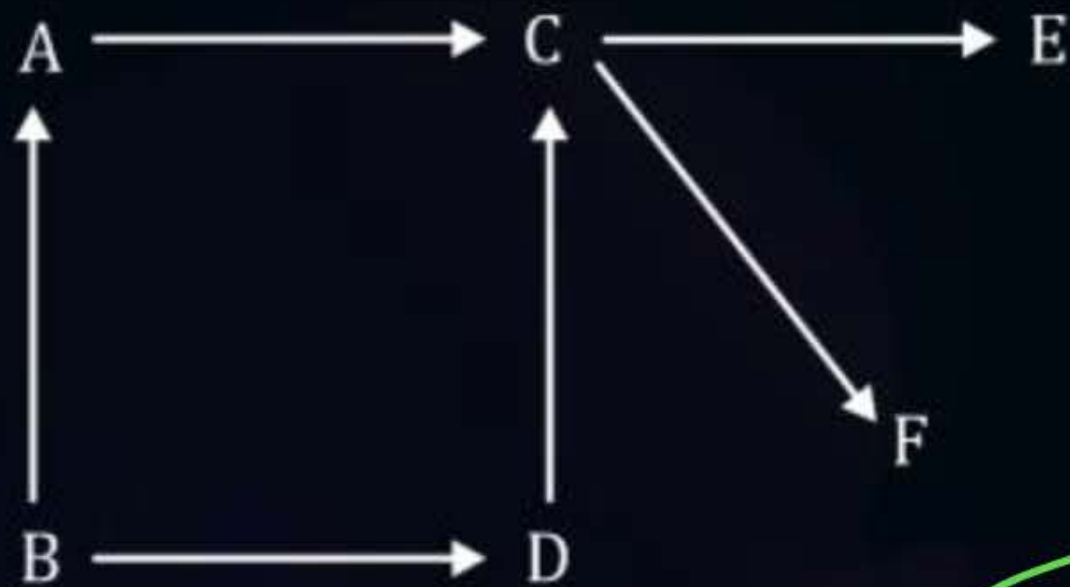


Topic : Graph Algorithms

Case-2:

DFS Starting at E

DAG



Finishing times

A → 9	D → 11
B → 12	D → 2
C → 6	F → 5

BDACFE

1/2

E

3/6

C

4/5

F

DFS Spanning forest

7/12

B

8/9

A

D

10/11

Decreasing order:

BDACFE

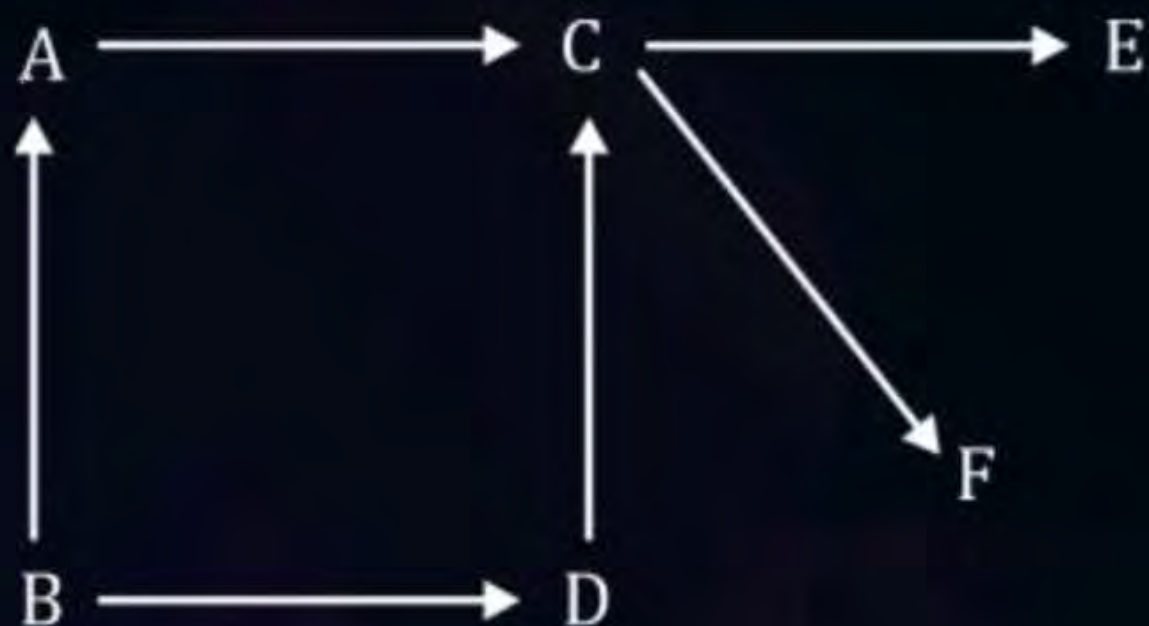


Topic : Graph Algorithms

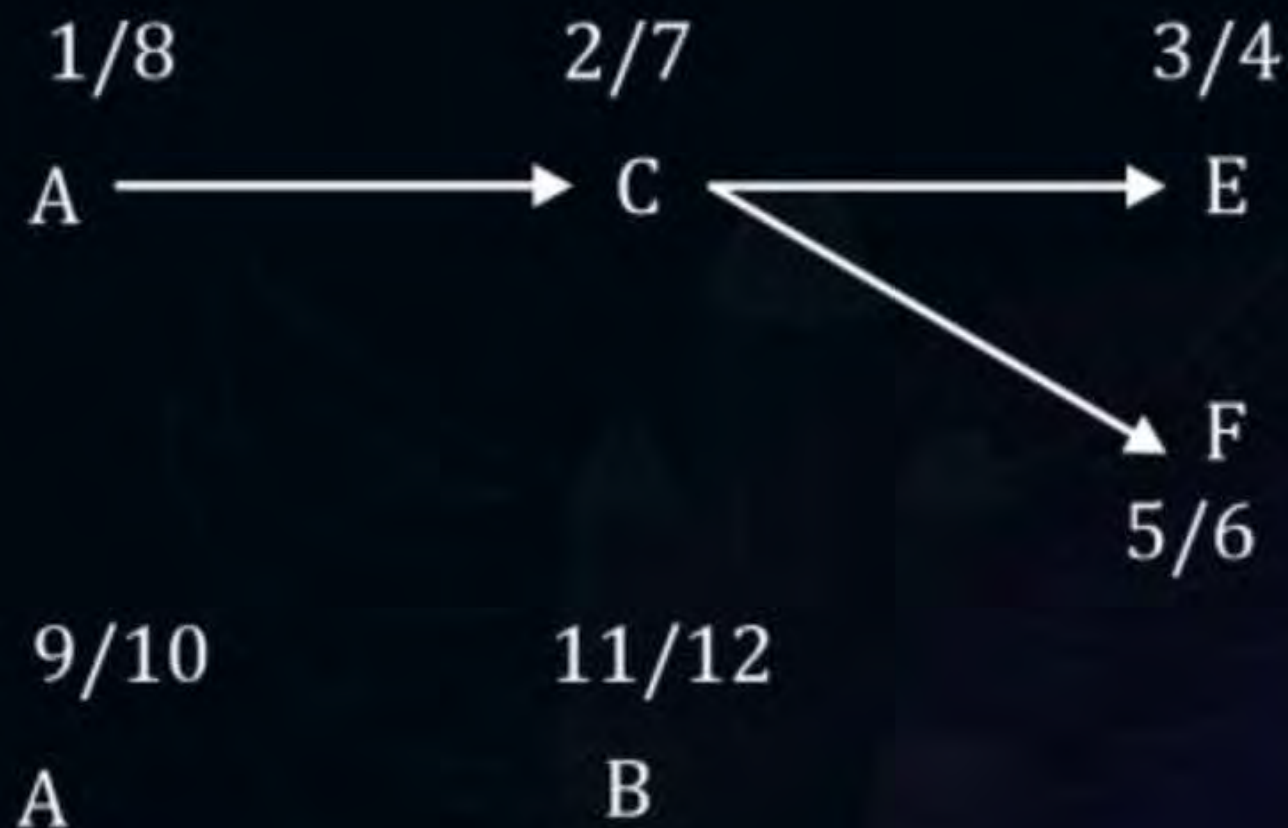
Case-3:

HW

Starting at D:



BADCFE





Topic : Graph Algorithms

HW

PYQ

#Q. Consider the depth-first-search of an undirected graph with 3 vertices P, Q, & R. Let discovery time $d(V)$ represent the time instant when the vertex 'V' is visited first, and finish time $f(V)$ represent finishing time ; given

$$d(p) = 5 \quad f(p) = 12$$

$$d(Q) = 6 \quad f(Q) = 10$$

$$d(R) = 14 \quad f(R) = 18$$

Which is true?

A

There is only one connected component.

B

There are two connected components, P and R are connected

C

There are two connected components, Q and R are

D

There are two connected components, P and Q are connected



THANK - YOU